

Date:-11|05|2026

Program 1**Linear Regression on California Housing Dataset**

Build a linear regression model on the California Housing dataset. Learn feature scaling, train-test split, and evaluate using MSE, RMSE, and R^2 score.

AI-Lab-6th-Sem-NIT-Srinagar > AI_lab_09 > Q1.py > ...

```
1  # Program 1: Linear Regression on California Housing Dataset
2  from sklearn.datasets import fetch_california_housing
3  from sklearn.model_selection import train_test_split
4  from sklearn.preprocessing import StandardScaler
5  from sklearn.linear_model import LinearRegression
6  from sklearn.metrics import mean_squared_error, r2_score
7  import matplotlib.pyplot as plt
8  import numpy as np
9  # Load dataset
10 data = fetch_california_housing()
11 X = data.data
12 y = data.target
13 print("Feature matrix shape :", X.shape)
14 # Split dataset
15 X_train, X_test, y_train, y_test = train_test_split(
16     X, y, test_size=0.2, random_state=42
17 )
18 # Feature scaling
19 scaler = StandardScaler()
20 X_train = scaler.fit_transform(X_train)
21 X_test = scaler.transform(X_test)
22 # Train model
23 model = LinearRegression()
24 model.fit(X_train, y_train)
25 # Predictions
26 y_pred_train = model.predict(X_train)
27 y_pred_test = model.predict(X_test)
28 # Metrics
29 train_mse = mean_squared_error(y_train, y_pred_train)
30 test_mse = mean_squared_error(y_test, y_pred_test)
31 rmse = np.sqrt(test_mse)
32 r2 = r2_score(y_test, y_pred_test)
33 print("Training MSE :", train_mse)
34 print("Testing MSE :", test_mse)
35 print("Testing RMSE :", rmse)
36 print("Testing R2 :", r2)
37 # Plot
38 plt.scatter(y_test, y_pred_test)
39 plt.xlabel("Actual Values")
40 plt.ylabel("Predicted Values")
41 plt.title("Actual vs Predicted")
42 plt.show()
```

ModuleNotFoundError: No module named 'sklearn'

(venv) gauravmathur@Annus-Mac AI_lab_09 % python3.11 Q1.py

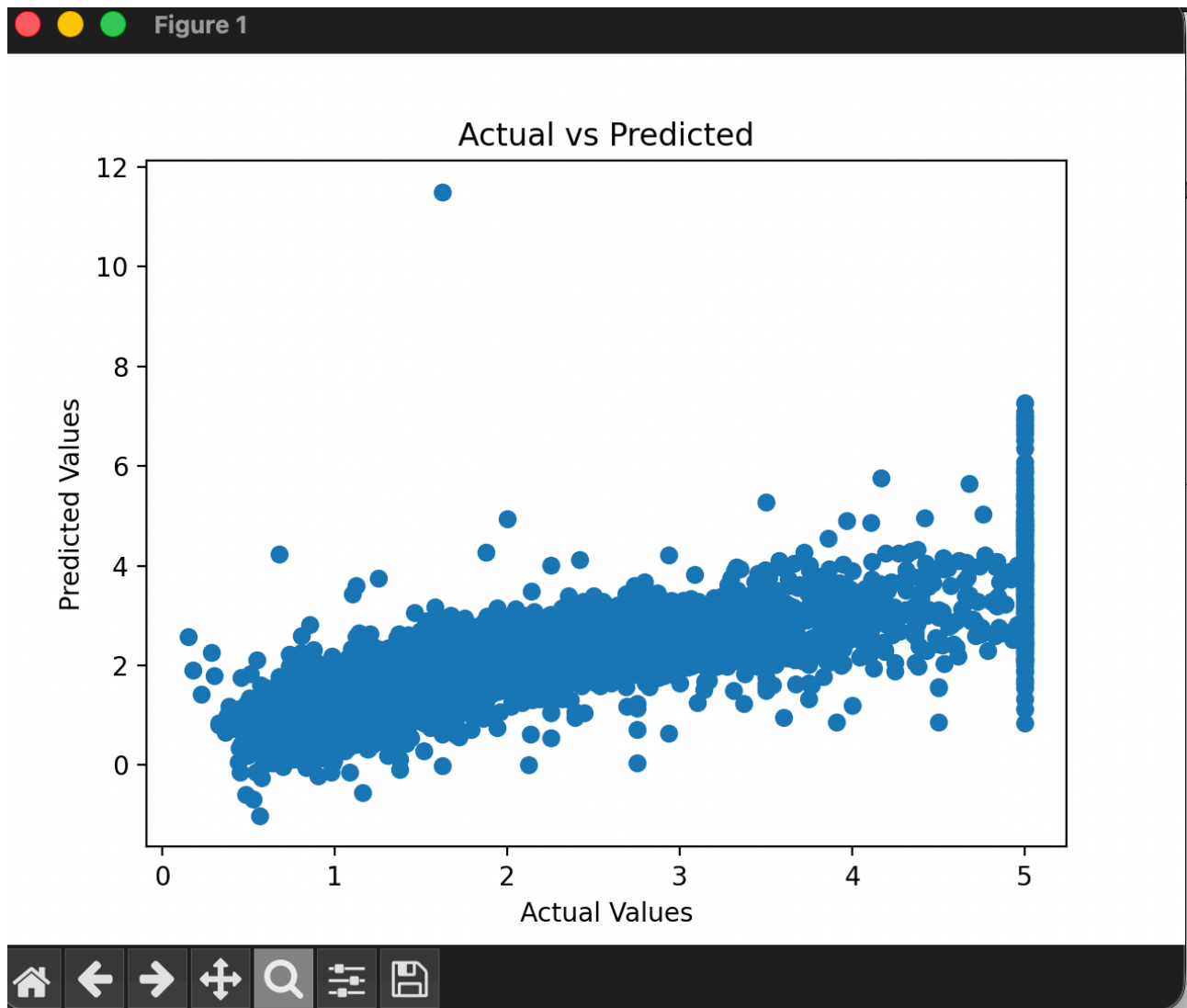
Feature matrix shape : (20640, 8)

Training MSE : 0.5179331255246699

Testing MSE : 0.5558915986952441

Testing RMSE : 0.7455813830127762

Testing R2 : 0.575787706032451



Program 2

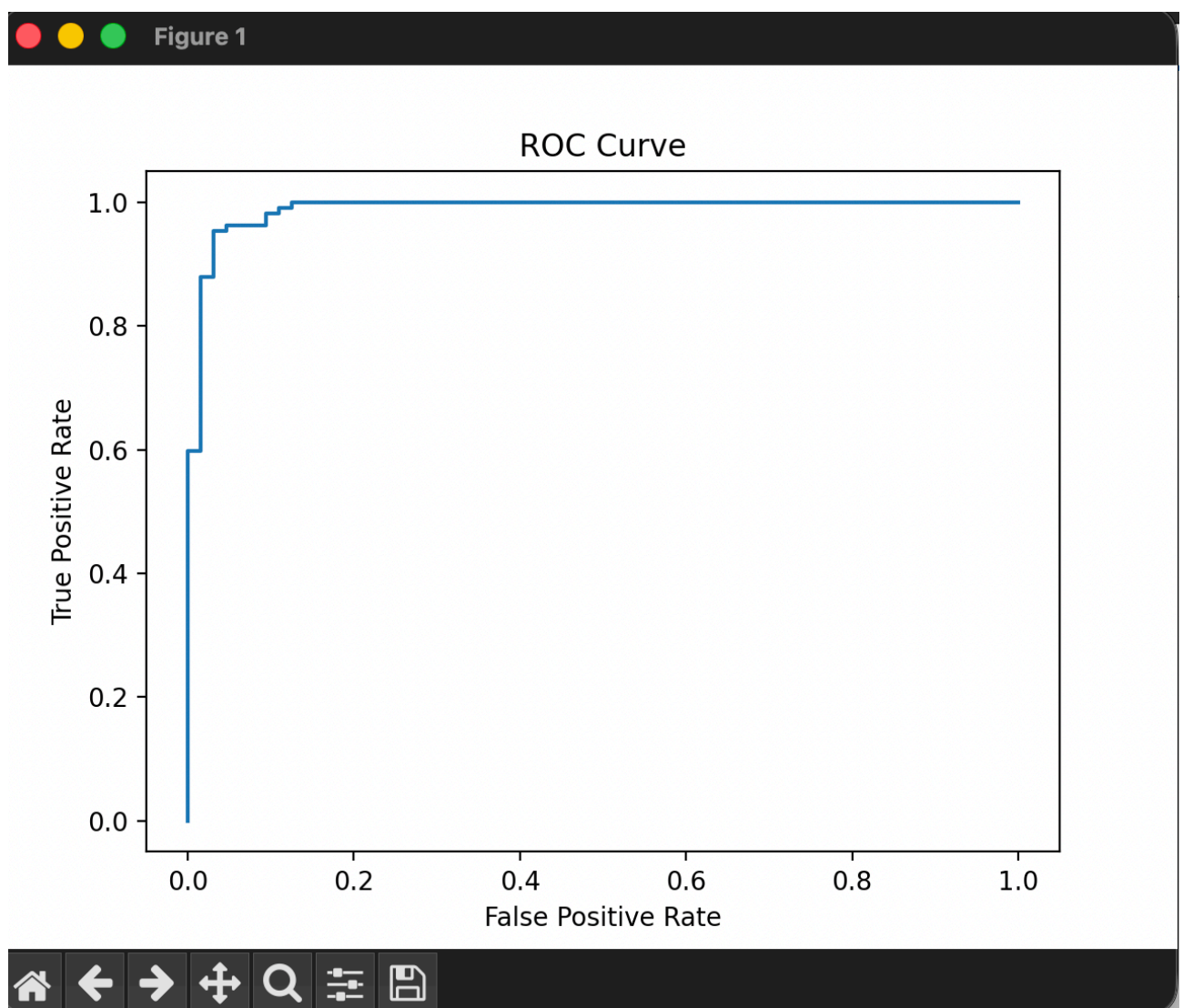
Logistic Regression for Binary Classification

Use logistic regression on the Iris or Breast Cancer dataset. Plot the confusion matrix, ROC curve, and visualize the decision boundary.

AI-Lab-6th-Sem-NIT-Srinagar > AI_lab_09 > Q2.py > ...

```
1  # Program 2: Logistic Regression for Binary Classification
2  from sklearn.datasets import load_breast_cancer
3  from sklearn.model_selection import train_test_split
4  from sklearn.linear_model import LogisticRegression
5  from sklearn.metrics import (
6      confusion_matrix,
7      accuracy_score,
8      precision_score,
9      recall_score,
10     f1_score,
11     roc_auc_score,
12     roc_curve
13 )
14 import matplotlib.pyplot as plt
15 # Load dataset
16 data = load_breast_cancer()
17 X = data.data
18 y = data.target
19 # Split dataset
20 X_train, X_test, y_train, y_test = train_test_split(
21     X,
22     y,
23     test_size=0.3,
24     stratify=y,
25     random_state=42
26 )
27 # Train model
28 model = LogisticRegression(max_iter=5000)
29 model.fit(X_train, y_train)
30 # Predictions
31 y_pred = model.predict(X_test)
32 # Probability scores
33 y_prob = model.predict_proba(X_test)[:, 1]
34 # Metrics
35 cm = confusion_matrix(y_test, y_pred)
36 accuracy = accuracy_score(y_test, y_pred)
37 precision = precision_score(y_test, y_pred)
38 recall = recall_score(y_test, y_pred)
39 f1 = f1_score(y_test, y_pred)
40 roc_auc = roc_auc_score(y_test, y_prob)
41 print("Confusion Matrix:")
42 print(cm)
43 print("Accuracy :", accuracy)
44 print("Precision :", precision)
45 print("Recall :", recall)
46 print("F1 Score :", f1)
47 print("ROC-AUC :", roc_auc)
48 # ROC Curve
49 fpr, tpr, thresholds = roc_curve(y_test, y_prob)
50 plt.plot(fpr, tpr)
51 plt.xlabel("False Positive Rate")
52 plt.ylabel("True Positive Rate")
53 plt.title("ROC Curve")
54 plt.show()
```

```
(venv) gauravmathur@Annus-Mac AI_lab_09 % python3.11 Q2.py
Confusion Matrix:
[[ 57   7]
 [  2 105]]
Accuracy : 0.9473684210526315
Precision : 0.9375
Recall : 0.9813084112149533
F1 Score : 0.958904109589041
ROC-AUC : 0.9889018691588785
```



Program 3

MNIST Digit Classification using Multiple Classifiers

Load the MNIST handwritten digits dataset and compare Logistic Regression, SVM, and Random Forest classifiers using accuracy, precision, recall, and F1-score.

AI-Lab-6th-Sem-NIT-Srinagar > AI_lab_09 > Q3.py > ...

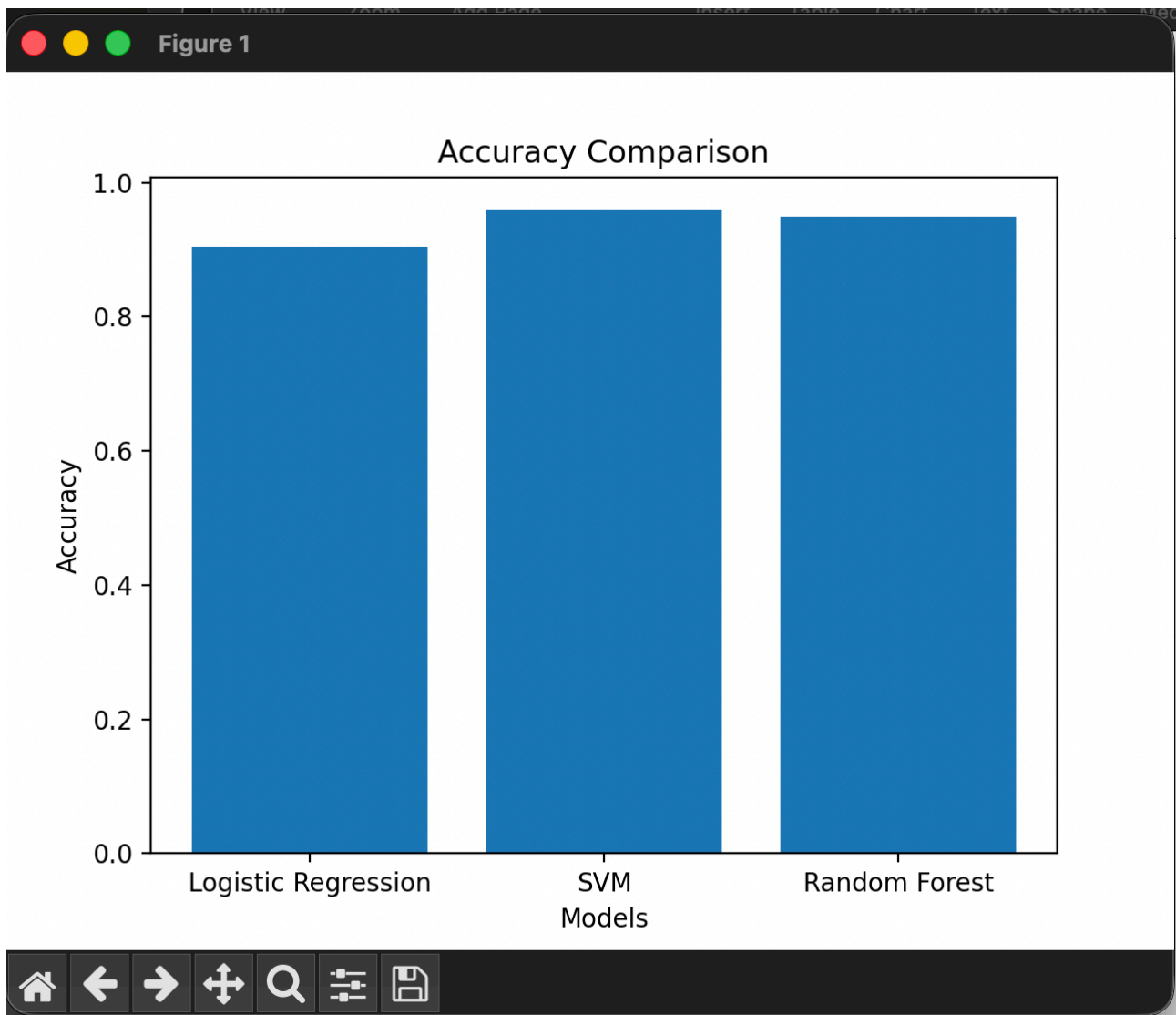
```
1  # Program 3: MNIST Digit Classification
2  from keras.datasets import mnist
3  from sklearn.linear_model import LogisticRegression
4  from sklearn.svm import SVC
5  from sklearn.ensemble import RandomForestClassifier
6  from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
7  import matplotlib.pyplot as plt
8  # Load dataset
9  (X_train, y_train), (X_test, y_test) = mnist.load_data()
10 # Normalize data
11 X_train = X_train / 255.0
12 X_test = X_test / 255.0
13 # Flatten images
14 X_train = X_train.reshape(60000, 784)
15 X_test = X_test.reshape(10000, 784)
16 # Reduce size for faster SVM training
17 X_train_small = X_train[:10000]
18 y_train_small = y_train[:10000]
19 # Models
20 lr = LogisticRegression(max_iter=1000)
21 svm = SVC(kernel='rbf')
22 rf = RandomForestClassifier()
23 # Train models
24 lr.fit(X_train_small, y_train_small)
25 svm.fit(X_train_small, y_train_small)
26 rf.fit(X_train_small, y_train_small)
27 # Predictions
28 lr_pred = lr.predict(X_test)
29 svm_pred = svm.predict(X_test)
30 rf_pred = rf.predict(X_test)
31 # Function for metrics
32 def evaluate(name, y_true, y_pred):
33     acc = accuracy_score(y_true, y_pred)
34     pre = precision_score(y_true, y_pred, average='macro')
35     rec = recall_score(y_true, y_pred, average='macro')
36     f1 = f1_score(y_true, y_pred, average='macro')
37     print("\n", name)
38     print("Accuracy :", acc)
39     print("Precision :", pre)
40     print("Recall :", rec)
41     print("F1 Score :", f1)
42     return acc
43 # Evaluate
44 acc1 = evaluate("Logistic Regression", y_test, lr_pred)
45 acc2 = evaluate("SVM", y_test, svm_pred)
46 acc3 = evaluate("Random Forest", y_test, rf_pred)
47 # Bar chart
48 models = ["Logistic Regression", "SVM", "Random Forest"]
49 accuracies = [acc1, acc2, acc3]
50 plt.bar(models, accuracies)
51 plt.xlabel("Models")
52 plt.ylabel("Accuracy")
53 plt.title("Accuracy Comparison")
54 plt.show()
```

```
(venv) gauravmathur@Annus-Mac AI_lab_09 % python3.11 Q3.py
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 4s 0us/step
```

```
Logistic Regression
Accuracy : 0.9044
Precision : 0.9032422787940598
Recall : 0.9031035265601683
F1 Score : 0.9030656432733986
```

```
SVM
Accuracy : 0.9594
Precision : 0.9593546447971855
Recall : 0.9590100865954783
F1 Score : 0.959091718039289
```

```
Random Forest
Accuracy : 0.9496
Precision : 0.9494167801881017
Recall : 0.9489830156318853
F1 Score : 0.9491154553455854
□
```



Program 4

Multiclass Classification — SVM, Random Forest & Neural Networks

Apply and compare three classifiers on the Iris or Wine dataset. Measure accuracy, macro F1-score, and training time; discuss trade-offs.

AI-Lab-6th-Sem-NIT-Srinagar > AI_lab_09 > Q4.py > ...

```
1  # Program 4: Multiclass Classification
2  from sklearn.datasets import load_wine
3  from sklearn.model_selection import train_test_split
4  from sklearn.preprocessing import StandardScaler
5  from sklearn.svm import SVC
6  from sklearn.ensemble import RandomForestClassifier
7  from sklearn.neural_network import MLPClassifier
8  from sklearn.metrics import accuracy_score, f1_score
9  import matplotlib.pyplot as plt
10 import time
11 # Load dataset
12 data = load_wine()
13 X = data.data
14 y = data.target
15 # Split dataset
16 X_train, X_test, y_train, y_test = train_test_split(
17     X,
18     y,
19     test_size=0.25,
20     random_state=42
21 )
22 # Standardization
23 scaler = StandardScaler()
24 X_train = scaler.fit_transform(X_train)
25 X_test = scaler.transform(X_test)
26 # Models
27 models = {
28     "SVM": SVC(kernel='rbf'),
29     "Random Forest": RandomForestClassifier(),
30     "MLP": MLPClassifier(max_iter=1000)
31 }
32 accuracies = []
33 f1_scores = []
34 times = []
35 # Train and evaluate
36 for name, model in models.items():
37     start = time.time()
38     model.fit(X_train, y_train)
39     end = time.time()
40     y_pred = model.predict(X_test)
41     acc = accuracy_score(y_test, y_pred)
42     f1 = f1_score(y_test, y_pred, average='macro')
43     training_time = end - start
44     accuracies.append(acc)
45     f1_scores.append(f1)
46     times.append(training_time)
47     print("\n", name)
48     print("Accuracy :", acc)
49     print("F1 Score :", f1)
50     print("Training Time :", training_time)
51 # Plot Accuracy
52 plt.bar(models.keys(), accuracies)
53 plt.xlabel("Models")
54 plt.ylabel("Accuracy")
55 plt.title("Accuracy Comparison")
56 plt.show()
```

```
Training Time : 0.0004290033090510
• (venv) gauravmathur@Annus-Mac AI_lab_09 % python3.11 Q4.py

SVM
Accuracy : 0.9777777777777777
F1 Score : 0.9764982373678026
Training Time : 0.0005879402160644531

Random Forest
Accuracy : 1.0
F1 Score : 1.0
Training Time : 0.04395294189453125

MLP
Accuracy : 0.9777777777777777
F1 Score : 0.9771428571428572
Training Time : 0.055948734283447266
○ (venv) gauravmathur@Annus-Mac AI_lab_09 %
```

